

Ideax – 2024 (IntelliAI Support Bot)

Front End code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>IntelliAI Chatbot</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f4;
      margin: 0;
      padding: 0;
    }

    .chat-container {
      max-width: 600px;
      margin: auto;
      padding: 20px;
      display: flex;
      flex-direction: column;
      height: 90vh;
      background-color: #fff;
      border-radius: 8px;
      box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
    }

    .chat-box {
      flex: 1;
      overflow-y: auto;
      padding: 20px;
      border: 1px solid #ddd;
      border-radius: 8px;
      background-color: #f9f9f9;
      margin-bottom: 10px;
      display: flex;
      flex-direction: column;
    }

    .message {
      margin-bottom: 10px;
    }

    .message.bot {
      text-align: left;
    }

    .message.user {
```

```
        text-align: right;
    }

    .message.bot p, .message.user p {
        display: inline-block;
        padding: 10px;
        border-radius: 8px;
        max-width: 80%;
    }

    .message.bot p {
        background-color: #e0e0e0;
    }

    .message.user p {
        background-color: #007bff;
        color: white;
    }

    .input-group {
        display: flex;
        gap: 10px;
        margin-top: 10px;
    }

    #text-input, #confirmation-input, #manager-email-input {
        flex: 1;
        padding: 10px;
        border-radius: 8px;
        border: 1px solid #ddd;
    }

    button {
        padding: 10px;
        border-radius: 8px;
        border: 1px solid #ddd;
        background-color: #007bff;
        color: white;
        cursor: pointer;
        transition: background-color 0.3s;
    }

    button:hover {
        background-color: #0056b3;
    }

    .section {
        display: none;
    }

    .section.active {
        display: block;
    }
```

```

    }
  </style>
</head>
<body>
  <div class="chat-container">
    <div class="chat-box" id="chat-box"></div>

    <div id="email-section" class="section active">
      <input type="email" id="email-input" placeholder="Your
login id">
      <button onclick="submitEmail()">Submit</button>
    </div>

    <div id="query-section" class="section">
      <div class="input-group">
        <input type="text" id="text-input"
placeholder="Type your message here">
        <input type="file" id="file-input">
        <button onclick="sendMessage()">Send</button>
      </div>
    </div>

    <div id="confirmation-section" class="section">
      <div class="input-group">
        <input type="text" id="confirmation-input"
placeholder="Type 'yes' to confirm or 'no' to cancel">
        <button id="confirm-button"
onclick="confirmAction()">Confirm</button>
      </div>
    </div>

    <div id="manager-email-section" class="section">
      <input type="email" id="manager-email-input"
placeholder="Manager's email">
      <button onclick="sendEmail()">Send Email</button>
    </div>

    <div id="more-requests-section" class="section">
      <div class="input-group">
        <input type="text" id="more-requests-input"
placeholder="Type your request here">
        <button id="check-more-requests-button"
onclick="checkMoreRequests()">Submit</button>
      </div>
    </div>
  </div>

  <script>
    let currentConfirmationMessage = '';
    let userid = '';
    let role = '';
    let role_gop = '';
    function addMessage(sender, message) {
    const chatBox = document.getElementById('chat-box');

```

```

const messageDiv = document.createElement('div');
messageDiv.className = `message ${sender}`;

// Replace '\n' with <br> to render line breaks in HTML
message = message.replace(/\n/g, '<br>');

// messageDiv.innerHTML = `<p>${sender === 'user' ? 'You: ' :
'Chatbot: '}${message}</p>`;
messageDiv.innerHTML = `<p>${message}</p>`;
chatBox.appendChild(messageDiv);
chatBox.scrollTop = chatBox.scrollHeight;
}

```

```

function submitEmail() {
    userid = document.getElementById('email-
input').value.trim();

    if (userid === "") {
        alert("Please enter your email address.");
        return;
    }

    addMessage('user', 'User id: ' + userid);
    document.getElementById('email-
section').classList.remove('active');
    document.getElementById('query-
section').classList.add('active');
    addMessage('bot', 'How can I assist you today?');
}

```

```

function sendMessage() {
const textInput = document.getElementById('text-input');
const fileInput = document.getElementById('file-input');
const file = fileInput.files[0];
const textMessage = textInput.value;

if (textMessage.trim() === "" && !file) {
    return;
}

// Check if textMessage likely contains an SQL query

const isSQLQuery = textMessage.toLowerCase().includes("sql")
||
    textMessage.toLowerCase().includes("query") ||
    textMessage.toLowerCase().includes("extract");

if (textMessage.trim() !== "") {
    addMessage('user', textMessage);
}

```

```

        // Choose the appropriate endpoint based on whether it's
an SQL query
        const endpoint = isSQLQuery ? '/sql' : '/text';

        fetch(endpoint, {
            method: 'POST',
            headers: { 'Content-Type': 'application/x-www-form-
urlencoded' },
            body: new URLSearchParams({ 'textMessage': textMessage
})
        })
        .then(response => response.json())
        .then(data => {
            addMessage('bot', data.response_message);
            if (data.response_message.includes("No matching role
found") ||
                data.response_message.includes("No Portfolio
found") ||
                data.response_message.includes("Portfolio not
active") ||
                data.response_message.includes("No matching gop
found")) {
                addMessage('bot', 'Please check with Eliot
support. ');
            } else if
((data.response_message.toLowerCase().includes("roles") ||
data.response_message.toLowerCase().includes("role") ||
data.response_message.toLowerCase().includes("gops") ||
data.response_message.toLowerCase().includes("gop")) &&
(data.response_message.toLowerCase().includes("approval") ||
data.response_message.toLowerCase().includes("manager"))) {
                currentConfirmationMessage =
data.response_message;
                role = data.response_message;
                document.getElementById('query-
section').classList.remove('active');
                document.getElementById('confirmation-
section').classList.add('active');
                addMessage('bot', 'Do you want access? Please type
"yes" to confirm or "no" to cancel. ');
            }
        });
        textInput.value = '';
    }

    if (file) {
        addMessage('user', 'You attached an image. ');
    }

```

```

const formData = new FormData();
formData.append('image', file);

fetch('/image_to_text', {
  method: 'POST',
  body: formData
})
.then(response => response.json())
.then(data => {
  addMessage('bot', data.response_message);
  if (data.response_message.includes("No matching role
found") ||
      data.response_message.includes("No Portfolio
found") ||
      data.response_message.includes("Portfolio not
active") ||
      data.response_message.includes("No matching gop
found")) {
    addMessage('bot', 'Please check with Eliot
support. ');
  } else if
((data.response_message.toLowerCase().includes("roles") ||
data.response_message.toLowerCase().includes("role") ||
data.response_message.toLowerCase().includes("gops") ||
data.response_message.toLowerCase().includes("gop")) &&
(data.response_message.toLowerCase().includes("approval") ||
data.response_message.toLowerCase().includes("manager"))) {
    currentConfirmationMessage =
data.response_message;
    role = data.response_message;
    document.getElementById('query-
section').classList.remove('active');
    document.getElementById('confirmation-
section').classList.add('active');
    addMessage('bot', 'Do you want access? Please type
"yes" to confirm or "no" to cancel. ');
  }
});

// Reset the file input after processing
fileInput.value = '';
}

function confirmAction() {
  const confirmationInput =
document.getElementById('confirmation-
input').value.trim().toLowerCase();

```

```

        if (confirmationInput === 'yes') {
            addMessage('user', 'Yes');
            document.getElementById('confirmation-
section').classList.remove('active');
            // addMessage('bot', currentConfirmationMessage);
            // Call the /text_to_role endpoint
            fetch('/text_to_role', {
                method: 'POST',
                headers: { 'Content-Type': 'application/x-www-form-
urlencoded' },
                body: new URLSearchParams({'userid': userid,
'textMessage': currentConfirmationMessage })
            })
            .then(response => response.json())
            .then(data => {
                // Get the response message from the backend (role is
already identified)
                // let role_gop = data.response_message;
                role_gop = data.response_message;
                addMessage('bot', role_gop)
                // Display the role-specific message from the backend
                if (data.response_message.includes("Matching GOP:") ||
data.response_message.includes('Matching Role:')){
                    // addMessage('bot', role);
                    // Deactivate all other sections and reactivate
the query section
                    document.getElementById('manager-email-
section').classList.add('active');
                    addMessage('bot', 'Please enter your manager\'s
email id:');
                }
                else{
                    document.getElementById('confirmation-
section').classList.remove('active');
                    document.getElementById('query-
section').classList.add('active');
                    document.getElementById('confirmation-input').value = '';
                }
            })
            .catch(error => {
                console.error('Error:', error);
                addMessage('bot', 'There was an error processing your
request. Please try again later.');
```

request. Please try again later.');

```

                document.getElementById('confirmation-
section').classList.remove('active');
                document.getElementById('query-
section').classList.add('active');
                document.getElementById('confirmation-input').value = '';
            });
        } else if (confirmationInput === 'no') {
            addMessage('user', 'No');
```

```

        addMessage('bot', 'Action canceled.');
```

// Deactivate all other sections and reactivate the query section
 document.getElementById('confirmation-section').classList.remove('active');
 document.getElementById('query-section').classList.add('active');

// Clear input fields
 document.getElementById('confirmation-input').value = '';
 } else {
 addMessage('bot', 'Invalid response. Please type "yes" to confirm or "no" to cancel.');

}

function sendEmail() {
 const managerEmail = document.getElementById('manager-email-input').value.trim();
 if (!managerEmail) {
 alert('Please enter your manager\'s email.');

return;
 }

addMessage('user', 'Manager\'s email: ' + managerEmail);
 fetch('/send_email', {
 method: 'POST',
 headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
 body: new URLSearchParams({
 'userid': userid,
 'managerEmail': managerEmail,
 'role': role_gop
 // , 'content': role_gop
 })
 })
 .then(response => response.json())
 .then(data => {
 addMessage('bot', 'Email sent successfully.');

resetQueryForNextInteraction();
 });
 }

function resetQueryForNextInteraction() {
 // Reset fields and activate query section for the next request
 document.getElementById('confirmation-input').value = '';
 }


```

        document.getElementById('manager-email-input').value =
'';

        document.getElementById('manager-email-
section').classList.remove('active');
        document.getElementById('query-
section').classList.add('active');
    }

    function checkMoreRequests() {
        const moreRequestInput =
document.getElementById('more-requests-input').value.trim();
        if (moreRequestInput) {
            addMessage('user', moreRequestInput);
            sendMessage();
        }
    }

</script>
</body>
</html>

```

Backend code (langchain-2.py):

```

from PyPDF2 import PdfReader
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.text_splitter import CharacterTextSplitter
from langchain.vectorstores import FAISS
from langchain.chains.question_answering import load_qa_chain
from langchain.llms import OpenAI
from flask import url_for
from flask import Flask, request, jsonify, render_template
import base64
import requests
import smtplib
from email.mime.text import MIMEText
# from openai import OpenAI
import uuid
import openai
import json
import re
import os

with open('/Users/dhiveshakilan/Learning/Python/AI/
IntelliAISupport/key.json') as f:
    key=json.load(f)

os.environ["OPENAI_API_KEY"] = key['API_KEY']

file_path= '/Users/dhiveshakilan/Learning/Python/AI/
IntelliAISupport/Info Doc/'

```

```

# List of PDF files
pdf_files =
['AMGEO.pdf', 'Eliot.pdf', 'Error.pdf', 'Ole_feeding.pdf', 'role_access.pdf', 'gop_access.pdf', 'XDS.pdf']
# , 'Sample_schema.pdf']
# , 'Table_schema_dict.pdf'] # Add more PDF paths as
needed

# Concatenate all the content from multiple PDFs
raw_text = ''

# Loop through each PDF file
for pdf_file in pdf_files:
    pdf = file_path+pdf_file
    pdfreader = PdfReader(pdf)

    # Read and extract text from each page in the PDF
    for i, page in enumerate(pdfreader.pages):
        content = page.extract_text()
        if content:
            raw_text += content

# We need to split the text using Character Text Split such that
it should not increase token size
text_splitter = CharacterTextSplitter(
    separator = ".",
    chunk_size = 2000,
    chunk_overlap = 200,
    length_function = len,
)
texts = text_splitter.split_text(raw_text)

# Download embeddings from OpenAI
embeddings = OpenAIEmbeddings()

document_search = FAISS.from_texts(texts, embeddings)

# document_search

chain = load_qa_chain(OpenAI(temperature = 0.2),
chain_type="stuff")

app = Flask(__name__)

user = {
    'login':
        {
            'akilan':
                {'active':'1', 'profile':'IT', 'mail':'dhivesh@gmail.com', 'sid':'da2024'},

```

```

        'suba':
{'active':'1','profile':'TS','mail':'suba@gmail.com','sid':'sb2023
'},
        'sanjay':
{'active':'0','profile':'IT','mail':'sanjay@gmail.com','sid':'sk20
22'},
        'shrishti':
{'active':'1','profile':'RDA','mail':'shrishti@gmail.com','sid':'s
h0404'}
    }
}
gop_perimeter = {
    'IT':{'Gop':{'PDS'},'Perimeter':{'ABD','BCD','KJH'}},
    'TS':{'Gop':{'',''},'Perimeter':{'ABD'}},
    'RD':{'Gop':{'*All*'},'Perimeter':{'BCD','KJH'}}
}
gop_data = {
    "PDS": {"Perimeter": "ABD","Status": "Active","RDB" :
"1234","Zone":"EUR"},
    "XYZ": {"Perimeter": "BCD","Status": "Active","RDB" :
"2345","Zone":"USD"},
    "FX": {"Perimeter": "KJH","Status": "Active","RDB" :
"3456","Zone":"HK"},
    "SNI": {"Perimeter": "BCD","Status": "Not Active","RDB" :
"4567","Zone":"EUR"},
    "J3": {"Perimeter": "KJH","Status": "Active","RDB" :
"5678","Zone":"EUR"}
}
ptf_data = [
{
    "PDS": [
        {"Portfolio": "PDS_ABC", "Status": "Active"},
        {"Portfolio": "PDS_XYZ", "Status": "Active"},
        {"Portfolio": "PDS_DEF", "Status": "Active"},
        {"Portfolio": "PDS_DEY", "Status": "Not Active"}
    ],
    "XYZ": [
        {"Portfolio": "XYZ_ABC", "Status": "Active"}
    ],
    "FX" : [
        {"Portfolio": "FX-GY-NOMGT", "Status": "Active"},
        {"Portfolio": "GY-MGT", "Status": "Not Active"}
    ],
    "SNI" : [
        {"Portfolio": "SN-FX-GY", "Status": "Not Active"}
    ],
    "J3" : [
        {"Portfolio": "JB-KJ-LI", "Status": "Active"}
    ]
}
]
roles_data = [

```

```

    {"Role": "DELETE_FLUX_X", "Description": "Delete flows via excel", "Keywords": "delete, flows, excel, not allowed to delete flows, no permission for deleting flows"},
    {"Role": "DELETE_OLD_DEAL_X", "Description": "Massive delete old deal x", "Keywords": "delete, old deals, massive delete, no permission for deleting deals, delete deal, not allowed to delete deals"},
    {"Role": "MAJ_FLOW_CVA", "Description": "Input of CVA Flows", "Keywords": "CVA, CVA flows, no access for CVA flows, unable to input CVA data"},
    {"Role": "MAJ_FLOW_OPERATIONEL", "Description": "Input of AJT, CLR, MTM Flows", "Keywords": "AJT flows, CLR flows, MTM flows, no permission for AJT, no access for CLR, unable to input MTM data, AJT, CLR, MTM"},
    {"Role": "MAJ_FLOW_TRESORERESULT", "Description": "Input of FOR, RES, SAF, FIN, EMB, EMD Flows", "Keywords": "treasury result flows, FOR, RES, SAF, FIN, EMB, EMD, no permission for treasury flows, not allowed to input FOR data"},
    {"Role": "MAJ_FLOW_TSF_SHB", "Description": "Input of TSF/SHB Flows", "Keywords": "TSF, TSF flows, SHB flows, no access to TSF/SHB, unable to input TSF/SHB data"},
    {"Role": "RESULTAT_VAL0", "Description": "Input of RHE, REM Flows", "Keywords": "RHE, RHE flows, REM flows, not allowed to input RHE/REM flows, no access for RHE/REM flows"},
    {"Role": "MAJ_FLOW_FLX", "Description": "Input of FLX Flows", "Keywords": "FLX, FLX flows, no permission for FLX flows, not allowed to input FLX data"},
    {"Role": "MAJ_FLOW_WHT", "Description": "Input of WHT Flows", "Keywords": "WHT, WHT flows, no access to WHT flows, unable to input WHT data"}
]
counterpart_data = []

```

schema_description = '''

Investment Banking Database Model Document

This document outlines the structure and relationships of the database model for tracking underlying securities, products derived from them, and deals made on these products. The database model consists of three main categories of tables: Underlying Tables, Product Tables, and Deal/Trade Tables.

1. Underlying Tables

These tables capture the details of basic financial instruments that can be used to create derivative products.

SHARE Table

Description: Stores information on shares.

Primary Key: share_id

Columns:

share_id (VARCHAR): Unique identifier for each share.

isin (VARCHAR): International Securities Identification Number for the share.

active (TINYINT): Flag indicating if the share is active (1 for active, 0 for inactive).

market_id (VARCHAR): Identifier for the market where the share is traded.

INDEX Table

Description: Stores information on indices.

Primary Key: index_id

Columns:

index_id (VARCHAR): Unique identifier for each index.

active (TINYINT): Flag indicating if the index is active (1 for active, 0 for inactive).

market_id (VARCHAR): Identifier for the market to which the index belongs.

2. Product Tables

Product tables store derivative products based on underlying assets such as shares and indices.

OPTION Table

Description: Stores option contracts, which can be based on shares or indices.

Primary Key: contract_id

Columns:

contract_id (INT): Unique identifier for each option contract.

underlying_id (VARCHAR): Identifier of the underlying asset, referencing either share_id from SHARE table or index_id from INDEX table.

contract_name (VARCHAR): Name of the option contract.

maturity (DATE): Maturity date of the option contract.

type (CHAR): Type of option (C for Call, P for Put).

Relationships:

underlying_id is a foreign key referencing either share_id in SHARE or index_id in INDEX based on the underlying asset.

FUTURE Table

Description: Stores future contracts, which can also be based on shares or indices.

Primary Key: contract_id

Columns:

contract_id (INT): Unique identifier for each future contract.

underlying_id (VARCHAR): Identifier of the underlying asset, referencing either share_id from SHARE table or index_id from INDEX table.

contract_name (VARCHAR): Name of the future contract.

maturity (DATE): Maturity date of the future contract.

Relationships:

underlying_id is a foreign key referencing either share_id in SHARE or index_id in INDEX based on the underlying asset.

3. Deal/Trade Tables

Deal tables store information on trades executed on products or directly on underlying securities. Each trade is versioned, allowing tracking of modifications and deletions.

SECURITY_DEALS Table

Description: Stores details of trades executed directly on shares, without involving derivative products.

Primary Key: deal_id

Columns:

deal_id (INT): Unique identifier for each security trade.

status (CHAR): Status of the trade (I for Initial, R for Recreated/Modified, D for Deleted, C for Cancelled).

date_modified (DATETIME): Timestamp of the most recent modification.

date_st (DATETIME): Timestamp indicating when this version of the trade was stopped. NULL for active trades.

trade_date (DATE): Execution date of the trade.

value_date (DATE): Date when the trade's value is realized.

trader_id (INT): Identifier for the trader executing the deal.

share_id (VARCHAR): Identifier of the underlying share from the SHARE table.

Relationships:

share_id is a foreign key referencing share_id in the SHARE table.

OPTION_DEALS Table

Description: Stores details of trades executed on options.

Primary Key: deal_id

Columns:

deal_id (INT): Unique identifier for each option trade.

status (CHAR): Status of the trade (same as above).

date_modified (DATETIME): Timestamp of the most recent modification.

date_st (DATETIME): Timestamp indicating when this version of the trade was stopped. NULL for active trades.

trade_date (DATE): Execution date of the trade.

value_date (DATE): Date when the trade's value is realized.

trader_id (INT): Identifier for the trader executing the deal.

contract_name (VARCHAR): Name of the contract, referencing contract_name in the OPTION table.

Relationships:

contract_name is a foreign key referencing contract_name in the OPTION table.

FUTURE_DEALS Table

Description: Stores details of trades executed on futures.

Primary Key: deal_id

Columns:

deal_id (INT): Unique identifier for each future trade.

status (CHAR): Status of the trade (same as above).

date_modified (DATETIME): Timestamp of the most recent modification.

date_st (DATETIME): Timestamp indicating when this version of the trade was stopped. NULL for active trades.

trade_date (DATE): Execution date of the trade.

value_date (DATE): Date when the trade's value is realized.

trader_id (INT): Identifier for the trader executing the deal.

contract_name (VARCHAR): Name of the contract, referencing contract_name in the FUTURE table.

Relationships:

contract_name is a foreign key referencing contract_name in the FUTURE table.

Versioning Workflow for Deal Tables

Initial Creation:

When a trade is created, status is set to I.

date_modified captures the creation timestamp.

date_st is set to NULL, indicating the trade is active.

Modification:

When a trade is modified, a new row is created with status set to R.

The previous version's date_st is updated to the modification timestamp.

The new version has date_modified set to the modification timestamp, with date_st as NULL.

Deletion:

When a trade is deleted, status is changed to D.

The date_st is set to the deletion timestamp.

Querying Active Trades:

Use WHERE date_st IS NULL to retrieve only active trades.

'''

```
def account(id):
    if id in user['login']:
        if user['login'][id]['active'] == '1':
            profile = user['login'][id]['profile']
            print(f'Profile {profile}')
            return ['Account is active' , profile]
        else:
            return ['Account is not active.']
    else:
        return ['No user id found.']

def gop_perimeter_check(gop, profile):
    print(gop, profile, gop_perimeter[profile]['Gop'],
gop_perimeter[profile])

    if profile in gop_perimeter:
        if gop in gop_perimeter[profile]['Gop']:
            return f'{gop} GOP access is already present for
profile {profile}.'

        else:
            gop_profile = gop_data[gop]['Perimeter']
            act_flg = gop_data[gop]['Status']

            if gop_profile in gop_perimeter[profile]['Perimeter']:
                if act_flg == 'Active':
                    return f'Matching GOP: {gop} is available for
profile: {profile}.'
                else:
                    return f'{gop} is in {gop_profile} perimeter
but is not active.'
            else:
```

```

        return f'User with profile {profile} does not have
access to the {gop_profile} perimeter for GOP {gop}.'

    else:
        return f'No matching profile found for {profile}.'

# Function to encode the image
def encode_image(image_path):
    with open(image_path, "rb") as image_file:
        return base64.b64encode(image_file.read()).decode('utf-8')

def langchain_doc(query):
    docs = document_search.similarity_search(query)
    r=chain.run(input_documents=docs, question=query)
    print('langchain',r)
    return r

# Function to match user query with role keywords using partial
matching
def match_role(role, roles_data):
    for i in range(len(roles_data)):
        if role.strip() in roles_data[i]['Role']:
            return f'{roles_data[i]['Role']}'
    return "No matching role found. - No description available"

def find_gop_using_ptf(ptf, ptf_data):
    found_gop=None
    print(ptf)
    for i,j in ptf_data[0].items():
        for k in range(len(j)):
            if j[k]['Portfolio'] in ptf.strip():
                found_gop = i if j[k]["Status"] == "Active" else
"Portfolio not active"
                break
    return found_gop if found_gop is not None else "Not found"

def mail(body,subject,sender_email,manager_email,sender_password):
    msg = MIMEText(body)
    msg['Subject'] = subject
    msg['From'] = sender_email
    msg['To'] = manager_email

    try:
        with smtplib.SMTP('smtp.gmail.com', 587) as server:
            server.starttls()
            server.login(sender_email, sender_password)
            server.send_message(msg)
            return jsonify({'status': 'success', 'message': 'Approval
email sent successfully.'})

    except Exception as e:
        return jsonify({'status': 'error', 'message': str(e)})

```



```

def counterpart(user_query, counterpart_data):
    # No counterpart Found
    return

# You have not the authorization to perform this action.
# (technical authorizationn: RUN with discrimnant 'SNIKO' on process
# group 'EVENT PROCESSING')

def generate_request_id():
    return str(uuid.uuid4())

def role_check(id,role):
    api_url = 'http://127.0.0.1:8080/getrole'
    response = requests.get(api_url)
    if response.status_code == 200:
        user_role = [req for req in response.json() if
req.get('user_id') == id]
        for i in user_role:
            if i['role']==role:
                return 'User already have access to this Role'
        return 'No role access given for this user'
    else:
        print(f"Failed to fetch requests: {response.status_code}")
        return []

def gop_check(id,gop):
    api_url = 'http://127.0.0.1:8080/getgop'
    response = requests.get(api_url)
    if response.status_code == 200:
        user_gop = [req for req in response.json() if
req.get('user_id') == id]
        for i in user_gop:
            if i['gop']==gop:
                return 'User already have access to this gop'
        return 'No gop access given for this user'
    else:
        print(f"Failed to fetch requests: {response.status_code}")
        return []

@app.route('/text', methods=['POST'])
def text():
    text_response = request.form.get('textMessage', '')
    k = langchain_doc(text_response)
    print(k)
    return jsonify({'response_message': k})

@app.route('/send_email', methods = ['POST'])
def send_approval_email():
    sender_email = "dhivesh18122000@gmail.com" # Replace with
your email
    sender_password = 'stwf byvp tuhv lskz'

```

```

role = request.form['role']
# content = request.form['content']
print(f'200 {role}')
user_id = request.form['userid']
manager_email = request.form['managerEmail']
request_id = generate_request_id()
words_after_colon = re.findall(r':\s*(\w+)', role)
if 'role' in role.lower() or 'roles' in role.lower():
    print(words_after_colon[0])
    rg = 'role'
    subject = f"Approval Request for {rg} access"
    url=f'https://docs.google.com/forms/d/e/
1FAIpQLScC4AvYbx9GIolMmjQQn62mfItYURw0mAZxUEyu4bWHN1ffNQ/viewform?
usp=pp_url&entry.1728767546={user_id}
&entry.1314294973={request_id}
&entry.1971473862={words_after_colon[0]}&entry.1503672414={rg}'
    body = f'''Hello,
        User {user_id} is requesting access to the
{words_after_colon[0]} {rg}.
        The request ID for this approval is {request_id}.
        Please review and provide your approval on the below
link.
        {url}

        Sincerely,
        Eliot Team'''
    return
mail(body,subject,sender_email,manager_email,sender_password)
elif 'gop' in role.lower() or 'gops' in role.lower() or
'portfolio' in role.lower() or 'portfolios' in role.lower():
    print('gop')
    print(words_after_colon[0],words_after_colon[1])
    rg = 'gop'
    profile=f'{rg}_{words_after_colon[1]}_Profile'
    subject = f"Approval Request for {rg} access"
    url=f'https://docs.google.com/forms/d/e/
1FAIpQLScC4AvYbx9GIolMmjQQn62mfItYURw0mAZxUEyu4bWHN1ffNQ/viewform?
usp=pp_url&entry.1728767546={user_id}
&entry.1314294973={request_id}
&entry.1971473862={words_after_colon[0]}
&entry.1503672414={profile}'
    body = f'''Hello,
        User {user_id} is requesting access to the
{words_after_colon[0]} {rg}.
        The request ID for this approval is {request_id}.
        Please review and provide your approval on the below
link.
        {url}

        Sincerely,
        Eliot Team'''

```

```

mail(body,subject,sender_email,manager_email,sender_password)
    team_email = 'dhivesh18122000@gmail.com'
    url2=f'https://docs.google.com/forms/d/e/
1FAIpQLScerQ3eeCRDpKobU-jP4x6WsY2XCVUdLJe0YfZEp_wGAkry_A/viewform?
usp=pp_url&entry.2116052852={user_id}
&entry.1558582620={request_id}
&entry.1060472253={words_after_colon[0]}
&entry.288713975={words_after_colon[1]}'
    subject = f"Approval Request for {rg} access"
    body2 = f'''Hello,
        Could you please provide approval to add
{words_after_colon[0]} {rg} to {words_after_colon[1]} profile.
        Please review and provide your approval on the below
link.
        {url2}
        Sincerely,
        Eliot Team'''
    return
mail(body2,subject,sender_email,team_email,sender_password)

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/sql', methods=['POST'])
def generate_response():
    # Get user input from the request
    user_input = request.form.get('textMessage', '')
    print(user_input)
    sql_keywords = ["SQL", "query", "database", "table", "column",
"join"]
    if not any(keyword.lower() in user_input.lower() for keyword
in sql_keywords):
        return "I don't know"

    # If SQL-related, proceed with generating the query
    prompt = f"""
Database Schema:
{schema_description}

User Request: {user_input}
"""

    try:
        # Use the updated OpenAI API method for completions
        completion = openai.chat.completions.create(
            model="gpt-4o-mini",
            messages=[
                {"role": "system", "content": "You are an expert
in SQL."},
                {"role": "user", "content": prompt}

```

```

        ]
    )
    return jsonify({'response_message':
completion.choices[0].message.content.strip()})

except Exception as e:
    return jsonify({'response_message': str(e)}), 500

@app.route('/image_to_text', methods=['POST'])
def extract_and_match():
    if 'image' not in request.files:
        return jsonify({"error": "No image part found"}), 400

    file = request.files['image']

    if file.filename == '':
        return jsonify({"error": "No image file selected"}), 400

    # Save the file temporarily
    file_path = '/tmp/' + file.filename
    file.save(file_path)

    base64_image = encode_image(file_path)

    headers = {
        "Content-Type": "application/json",
        "Authorization": f"Bearer {os.getenv('OPENAI_API_KEY')}"
    }

    payload = {
        "model": "gpt-4o-mini",
        "messages": [
            {
                "role": "user",
                "content": [
                    {
                        "type": "text",
                        "text": "Extract only the error message
text from this image."
                    },
                    {
                        "type": "image_url",
                        "image_url": {
                            "url": f"data:image/jpeg;base64,
{base64_image}"
                        }
                    }
                ]
            }
        ],
        "max_tokens": 300
    }

```

```

        response = requests.post("https://api.openai.com/v1/chat/
completions", headers=headers, json=payload)

        if response.status_code == 429:
            return jsonify({"error": "Rate limit exceeded. Please try
again later."}), 429

        if response.status_code != 200:
            return jsonify({"error": f"API request failed with status
code {response.status_code}", "details": response.json()}),
response.status_code

        try:
            response_json = response.json()
            user_query = response_json['choices'][0]['message']
['content']
        except KeyError:
            return jsonify({"error": "Failed to parse response",
"response": response.json()}), 500

        if not user_query:
            return jsonify({"error": "No message extracted"}), 400

        text_endpoint_url = request.host_url + 'text' # Constructs
the full URL for the /text endpoint
        text_response = requests.post(text_endpoint_url,
data={'textMessage': user_query})

        if text_response.status_code != 200:
            return jsonify({"error": "Failed to process text",
"details": text_response.json()}), text_response.status_code
        print(text_response.json())
        return text_response.json()

@app.route('/text_to_role', methods=['POST'])
def text_to_role():
    id = request.form.get('userid', '')
    text_message = request.form.get('textMessage', '')
    print(text_message)
    if not text_message:
        return jsonify({"error": "No text message provided"}), 400

    # query = "Why I am getting this message - "+text_message
    if 'role' in text_message.lower() or 'roles' in
text_message.lower():
        q = f"Only return the role name from this response:
{text_message}"
        matching = match_role(langchain_doc(q), roles_data)
        print(matching)
        r = role_check(id, matching)
        if r == 'No role access given for this user':

```

```

        response_message = f"Matching Role: {matching}"
        print(response_message)
    else:
        response_message = r
        elif 'portfolio' in text_message.lower() or 'portfolios' in
text_message.lower() or 'gop' in text_message.lower() or 'gops' in
text_message.lower():
            print('ptf')
            q1 = f"Extract and return only the `portfolio` name from
this response: {text_message}. Do not include any other text or
formatting."
            matching = find_gop_using_ptf(langchain_doc(q1), ptf_data)
            print(matching)
            r=gop_check(id,matching)
            if matching != "Portfolio not active" and matching != "Not
found" and r=='No gop access given for this user':
                print('first print')
                profile_id=account(id)
                if len(profile_id)>1:
                    profile = f"{profile_id[1]}"
                    print(profile)
                    response_message =
gop_perimeter_check(matching.strip(),profile)
                else:
                    response_message= profile_id[0]
            else:
                print('gop')
                q = f"Extract and return only the `gop` name from this
response: {text_message}. Do not include any other text or
formatting."
                matching = langchain_doc(q)
                # q1 = f"Extract and return only the `portfolio` name
from this response: {text_message}. Do not include any other text
or formatting."
                # matching_ptf = find_gop_using_ptf(langchain_doc(q),
gop_data)
                # print(matching_ptf)
                profile_id=account(id)
                print(matching)
                r=gop_check(id,matching)
                if len(profile_id)>1 and r=='No gop access given for
this user':
                    profile = f"{profile_id[1]}"
                    print(profile)
                    response_message =
gop_perimeter_check(matching.strip(),profile)
                else:
                    response_message= profile_id[0]
                # else:
                #     response_message = matching_ptf

        elif 'counterpart' in text_message.lower():

```

```

        matching = counterpart(text_message, counterpart_data)
        response_message = f"Eliot Code is: {matching}"

    else:
        response_message = f"{text_message}"
    print(response_message)

    return jsonify({
        "extracted_message": text_message,
        "response_message": response_message
    })

if __name__ == '__main__':
    app.run(debug=True)

```

SQL API request(main.py):

```

from flask import Flask, request, jsonify
from db import get_db_connection # Import the function to get
database connection
import os
import psycopg2

app = Flask(__name__)

@app.route('/')
def home():
    return "Welcome to the API!"

@app.route('/storegop', methods=['POST'])
def store_gop():
    data = request.json
    if not data:
        return jsonify({"message": "No data provided."}), 400

    user_id = data.get('user_id')
    profile = data.get('profile')
    gop_name = data.get('gop_name')
    perimeter = data.get('perimeter')
    read = data.get('read')
    write = data.get('write')

    if not user_id or not profile or not gop_name or not perimeter
or not read or not write:
        return jsonify({"message": "Missing data fields."}), 400

    conn = get_db_connection()
    cur = conn.cursor()
    cur.execute('''
        INSERT INTO gop
        (user_id,profile,gop_name,perimeter,read,write)

```

```

        VALUES (%s, %s, %s,%s, %s, %s)
        ''', (user_id,profile,gop_name,perimeter,read,write))
    conn.commit()
    cur.close()
    conn.close()

    return jsonify({"message": "Request information stored
successfully.", "data": data}), 200

@app.route('/getgop', methods=['GET'])
def get_gop():
    conn = get_db_connection()
    cur = conn.cursor()
    cur.execute('SELECT * FROM gop')
    rows = cur.fetchall()
    cur.close()
    conn.close()

    requests = []
    for row in rows:
        user_id,profile,gop_name,perimeter,read,write = row
        requests.append({
            'user_id': user_id,
            'profile': profile,
            'gop_name': gop_name,
            'perimeter': perimeter,
            'read': read,
            'write': write
        })

    return jsonify(requests), 200

@app.route('/storeroles', methods=['POST'])
def store_role():
    data = request.json
    if not data:
        return jsonify({"message": "No data provided."}), 400

    user_id = data.get('user_id')
    role = data.get('role')
    description = data.get('description')

    if not user_id or not role or not description:
        return jsonify({"message": "Missing data fields."}), 400

    conn = get_db_connection()
    cur = conn.cursor()
    cur.execute('''
        INSERT INTO role (user_id,role,description)
        VALUES (%s, %s, %s)
        ''', (user_id,role,description))
    conn.commit()

```



```

        cur.close()
        conn.close()

    return jsonify({"message": "Request information stored
successfully.", "data": data}), 200

@app.route('/getrole', methods=['GET'])
def get_role():
    conn = get_db_connection()
    cur = conn.cursor()
    cur.execute('SELECT * FROM role')
    rows = cur.fetchall()
    cur.close()
    conn.close()

    requests = []
    for row in rows:
        user_id, role, description = row
        requests.append({
            'user_id': user_id,
            'role': role,
            'description': description
        })

    return jsonify(requests), 200

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=int(os.environ.get('PORT',
8080)))

```

DB connection (db.py):

```

import psycopg2

# Database connection parameters
db_params = {
    'host': '34.93.11.95',
    'port': '5432',
    'database': 'postgres',
    'user': 'ltai',
    'password': 'msdthala2019'
}

def get_db_connection():
    return psycopg2.connect(**db_params)

# create_table_query = """
# CREATE TABLE IF NOT EXISTS Role (
#     user_id VARCHAR(255),
#     role VARCHAR(255),
#     description VARCHAR(255)
# );

```

```

# """"

# create_table_query = """"
# CREATE TABLE IF NOT EXISTS Gop (
#     user_id VARCHAR(255),
#     profile VARCHAR(255),
#     gop_name VARCHAR(255),
#     perimeter VARCHAR(255),
#     read VARCHAR(255),
#     write VARCHAR(255)
# );
# """"

# try:
#     # Connect to the PostgreSQL database
#     conn = get_db_connection()
#     cur = conn.cursor()

#     # Execute the create table query
#     cur.execute(create_table_query)

#     # Commit changes
#     conn.commit()

#     print("Table created successfully.")

#     # Close the cursor and connection
#     cur.close()
#     conn.close()

# except Exception as e:
#     print(f"Error: {e}")

# try:
#     conn = get_db_connection()
#     cur = conn.cursor()

#     # Execute the delete statement
#     cur.execute('DELETE FROM requests')

#     # Commit the transaction
#     conn.commit()

# except Exception as e:
#     print(f"An error occurred: {e}")
#     # Optionally, roll back the transaction if something went
#     wrong
#     conn.rollback()

# finally:
#     # Close the cursor and connection in the finally block to
#     ensure they are always closed

```

```
# cur.close()
# conn.close()
```

Access Granting after approvals (access_granting_form.py):

```
import pandas as pd
import os
import requests

# File paths
processed_entries_file = '/Users/dhiveshakilan/Learning/Python/AI/IntelliAISupport/processed_entries.csv'
role_gop_manager_approval = 'https://docs.google.com/spreadsheets/d/e/2PACX-1vS30w9LeJpEfTxG20LVi9LMJUfe961y5tg2ljrFuth_0g3SkeLiSwpIMSU7Blc7HnuL1RxdaHmv0Ekq/pub?output=csv'
gop_approval = 'https://docs.google.com/spreadsheets/d/e/2PACX-1vTCBhZST67T28MZH7_D7es_IExPaV9I8M4wJTHiYCDW0cLRm5Iqw3zVMC-7nC273JQmtktnC8LW0Ny1/pub?output=csv'

# Read CSV data from Google Sheets
role_gop_manager_approval_data =
pd.read_csv(role_gop_manager_approval)
gop_approval_data = pd.read_csv(gop_approval)

# Load the processed tickets with deduplication
if os.path.exists(processed_entries_file):
    processed_entries = pd.read_csv(processed_entries_file)
    processed_tickets = processed_entries['Ticket
No'].drop_duplicates().tolist()
else:
    processed_tickets = []

# Filter to include only unprocessed rows
df_role_manager = role_gop_manager_approval_data[
    (role_gop_manager_approval_data['Do you approve access to the
User Specific Role/GOP?'] == 'Yes, I approve.') &
    (role_gop_manager_approval_data['Description'] == 'role') &
    (~role_gop_manager_approval_data['Ticket
No'].isin(processed_tickets))
]
df_gop_manager = role_gop_manager_approval_data[
    (role_gop_manager_approval_data['Do you approve access to the
User Specific Role/GOP?'] == 'Yes, I approve.') &

(role_gop_manager_approval_data['Description'].str.contains('gop',
case=False, na=False)) &
    (~role_gop_manager_approval_data['Ticket
No'].isin(processed_tickets))
]
df_gop = gop_approval_data[
```

```

        (gop_approval_data['Do you approve to add this gop in
mentioned profile?'] == 'Yes, I approve.') &
        (~gop_approval_data['Ticket No'].isin(processed_tickets))
    ]
    # print(df_gop)
    # print(df_gop_manager)
    # Process entries and keep track of newly processed tickets
    newly_processed_tickets = []

    api_url = 'http://127.0.0.1:8080'

    # Grant access for roles
    for _, row in df_role_manager.iterrows():

        url = api_url+'/storeroles'
        data = {
            'user_id': row['User Name'],
            'role': row['Role/Gop Name'],
            'description': row['Description']
        }
        print(data)
        try:
            response = requests.post(url, json=data)
            print(response)
            if response.status_code == 200:
                print(f"Request information stored successfully.")
                print('Access is granted for role manager:',
row['Ticket No'])
            else:
                print(f"Failed to store request information:
{response.status_code}")
            except requests.exceptions.RequestException as e:
                print(f"Error storing request information: {e}")

        newly_processed_tickets.append(row['Ticket No'])

    # Merge and process GOP approvals
    matched_df = pd.merge(df_gop_manager, df_gop, on='Ticket No',
how='inner')
    print(matched_df)
    for _, row in matched_df.iterrows():
        print(row['Do you approve to add this gop in mentioned
profile?'], row['Do you approve access to the User Specific Role/
GOP?'])
        data = {
            'user_id': row['User Name_x'],
            'profile': row['Profile'],
            'gop_name': row['GOP Name'],
            'perimeter': row['Description'],
            'read': '1',
            'write': '1'

```

```

    }
    print(data)
    if row['Do you approve to add this gop in mentioned profile?']
== row['Do you approve access to the User Specific Role/GOP?']:
        url = api_url+'/storegop'
        data = {
            'user_id': row['User Name_x'],
            'profile': row['Profile'],
            'gop_name': row['GOP Name'],
            'perimeter': row['Description'],
            'read': '1',
            'write': '1'
        }
        print(data)
        try:
            response = requests.post(url, json=data)
            print(response)
            if response.status_code == 200:
                print(f"Request information stored successfully.")
                print('Access is granted for GOP:', row['Ticket
No'])
            else:
                print(f"Failed to store request information:
{response.status_code}")
                except requests.exceptions.RequestException as e:
                    print(f"Error storing request information: {e}")

            newly_processed_tickets.append(row['Ticket No'])

# Deduplicate newly processed tickets
newly_processed_tickets = list(set(newly_processed_tickets))

# Update processed_entries.csv with new Ticket Nos
if newly_processed_tickets:
    # Create a DataFrame for the new entries
    new_entries_df = pd.DataFrame(newly_processed_tickets,
columns=['Ticket No'])

    # Append without duplicating headers and prevent duplicate
entries
    if os.path.exists(processed_entries_file):
        existing_df = pd.read_csv(processed_entries_file)
        updated_df = pd.concat([existing_df,
new_entries_df]).drop_duplicates()
        updated_df.to_csv(processed_entries_file, index=False)
    else:
        new_entries_df.to_csv(processed_entries_file, index=False,
header=True)

```

The code provided could be categorized under Retrieval-Augmented Generation (RAG), as it includes elements that enhance responses by retrieving and processing data from specific sources before generating responses. In RAG applications, systems combine retrieval and generation to create richer, more relevant outputs, often using external knowledge sources or data that dynamically inform the responses.

RAG Concepts in the Code

- **Retrieval with LangChain**: The code includes ``langchain_doc(text_response)`` calls, indicating the use of LangChain to analyze and match queries with roles, portfolios, or counterparts. LangChain is commonly used in RAG to process and retrieve contextual data from unstructured or semi-structured sources, enhancing the application's ability to generate accurate responses based on previous information or specific text content.
- **Augmentation**: After extracting user queries or text from images, the app queries databases or performs partial matches to identify relevant role or portfolio information. This retrieved information, like matched roles or GOP (Group of Portfolios), is then combined with static text to generate dynamic email bodies or response messages for the users.
- **Generation**: The app uses OpenAI's GPT model (``gpt-4o-mini``) to enhance certain responses, especially when generating SQL queries or processing complex text queries. This could align with RAG's "generation" component, which takes retrieved data and augments it to craft final outputs.

The above code is a Python web application built using Flask, which facilitates handling and routing different types of user requests, such as sending approval emails, querying a database, and processing text and images. The application uses various libraries to perform tasks like sending emails, making HTTP requests, and handling data. Here's a breakdown of each part of the code, the libraries used, and the technologies that power it.

1. Technologies and Libraries Used

1. Flask:

- Flask is a lightweight Python web framework used to create web applications.
- Provides tools for routing HTTP requests, rendering templates, handling forms, and JSON responses.

2. Requests:

- Used for making HTTP requests to APIs.
- In this application, ``requests.get()`` is used to retrieve user roles and GOPs from API endpoints.

3. **smtpplib**:
 - Python's built-in library for sending emails.
 - Used here to connect to Gmail's SMTP server and send approval request emails.
4. **re (Regular Expressions)**:
 - Used to extract specific parts of text strings using patterns.
 - For instance, `re.findall(r':\s*(\w+)', role)` is used to extract words following a colon from a string.
5. **uuid**:
 - Generates unique identifiers, commonly used for request tracking.
 - Here, `uuid.uuid4()` is used to create a unique request ID for each approval request.
6. **OpenAI API (for chat completion and image processing)**:
 - The OpenAI API provides access to models for text generation and image processing.
 - Used here to generate SQL queries, extract text from images, and interpret text queries.

2. **Code Structure and Explanation**

Helper Functions

1. **match_role(role, roles_data)**:
 - Takes a `role` string and a list of roles (`roles_data`) and checks if the `role` partially matches any item in the list.
 - Returns the matched role or a default message if no match is found.
2. **find_gop_using_ptf(ptf, ptf_data)**:
 - Searches for a portfolio (`ptf`) in the `ptf_data` dictionary.
 - Returns the `gop` if it's found and active, or a default message if it's inactive or not found.
3. **mail(body, subject, sender_email, manager_email, sender_password)**:
 - Sends an email using Gmail's SMTP server.
 - Configures the email content and sends it to the recipient (manager) to request approval for access.
4. **generate_request_id()**:
 - Generates a unique UUID for each request to track approvals.
5. **role_check(id, role)**:

- Makes a GET request to the `/getrole` endpoint to verify if a user has access to a specific role.
- Returns a message indicating if access is granted or denied.

6. `**`gop_check(id, gop)`**`:

- Similar to `role\_check`, but checks for access to a `gop`.
- Returns access status or an error message if the request fails.

Flask Routes

1. `**`/text`**`:

- Processes a text message (`textMessage`) and forwards it to `langchain\_doc` for further processing.
- Returns the response as a JSON object.

2. `**`/send_email`**`:

- This route is used to send an email requesting approval.
- Checks if the request pertains to a `role` or `gop`, constructs the email content accordingly, and sends it to the manager.

3. `**`/sql`**`:

- This endpoint handles SQL-related queries.
- Checks if the input contains SQL keywords, and if so, forwards the query to the OpenAI API to generate SQL based on a provided schema.

4. `**`/image_to_text`**`:

- Extracts text from an image uploaded in a form.
- The image is saved temporarily, encoded to base64, and sent to OpenAI for text extraction.
- The extracted text is sent to `/text` for further processing.

5. `**`/text_to_role`**`:

- Interprets a text message to determine if it's related to `role`, `gop`, or counterpart access.
- Based on the message content, it calls helper functions like `role\_check` or `gop\_check` to validate access permissions.
- Returns a response indicating the requested information or access status.

6. `**`/`**`:

- Serves the main index page of the application.

How Each Part Works Together

1. ****User Inputs****: Users submit requests or images for processing through the respective routes.
2. ****Text and SQL Processing****:
 - For text requests, keywords determine if they're related to roles, `gop`, or SQL.
 - SQL queries are forwarded to OpenAI's language model to generate appropriate queries based on the schema.
3. ****Image-to-Text Extraction****:
 - Uploaded images are processed, text is extracted, and then the extracted message is forwarded to another endpoint for analysis.
4. ****Access Validation and Approval Workflow****:
 - Access requests are validated using `role\_check` and `gop\_check`.
 - Approval emails are sent to managers based on the result.
5. ****API Communication****:
 - The application communicates with an external API (at `127.0.0.1:8080`) to verify user roles and GOPs.
